実装詳細書 電力需要予測システム

最終更新: 2026-03-21 **対応バージョン:** 1.0.0 **ステータス:** Production Ready

---

開発ツール

コードの品質管理

Linter: ruff v0.3.0 **フォーマッタ:** ruff v0.3.0 (Black互換モード)

型チェック: mypy v1.8.0

pre-commit フック: trailing-whitespace, end-of-file-fixer, check-yaml, check-toml, check-ast, check-merge-conflict, check-large-files (1MB), debug-statements, mixed-line-ending, ruff, ruff-format, mypy

依存関係管理

- pyproject.toml で依存関係を宣言
- uv.lock でロックファイルを管理

開発コマンド

```
uv sync --dev                    # 開発環境のセットアップ
uv run ruff check .              # Lint
uv run ruff format --check .     # フォーマットチェック
uv run mypy src/                 # 型チェック
uv run pytest                    # テスト
uv run pre-commit run --all-files # 全フック実行
```

開発ツール

ツール	バージョン	用途
ruff	0.3.0	Lint + フォーマット
mypy	1.8.0	型チェック
pre-commit	3.6.0	Gitフック管理
pytest	8.0.0	テスト
pytest-cov	4.1.0	カバレッジ測定

CI/CD

ジョブ	ツール	実行内容
Lint	ruff	check + format --check
Type Check	mypy	型チェック
Test	pytest	ユニットテスト + カバレッジ
Security	pip-audit	脆弱性スキャン

実装原則

- **Protocol ベースの抽象化** — 継承ではなく構造的サブタイピング
- **ファイル単位の分離** — 1 ファイル = 1 クラス
- **TDD** — テストを先に書く
- **型ヒント必須** — mypy strict モード
- **ruff 設定** — E, W, F, I, N, UP, B, SIM, RUF ルール

ディレクトリ構成

```
src/
tests/
pyproject.toml
uv.lock
.pre-commit-config.yaml
```

コマンド

```
uv run pytest -v -s -x --cov=src --cov-report=term-missing
uv run ruff check --fix .
uv run ruff format .
uv run mypy src/
uv run pre-commit install
uv run pre-commit run --all-files
```

データ基盤

データ取得基盤の実装

状態: ✅ Done

概要

- Open-Meteo 気象データ API クライアント実装
- 気象データ変換モジュール実装（DataFrame 形式）
- 電力需要データ CSV 読み込み実装
- 統合データセット構築パイプライン実装

データ取得の仕様

状態: ✅ Done

概要

- Open-Meteo 気象データ API の仕様調査
- 対象地域・変数・期間の定義

データ変換・品質管理の仕様

状態: ✅ Done

概要

- 欠損値補完（線形補間）
- 外れ値検出（IQR 法）
- 時系列整合性チェック
- 学習用データセット構築

Phase 1 データ収集 API

状態: ✅ Done

概要

- Open-Meteo Historical Weather API（無料・APIキー不要）
- 対象地域: 東京（緯度 35.6895, 経度 139.6917）
- 対象期間: 2023-01-01 ～ 2024-12-31（2年分）
- 取得変数: temperature_2m, relative_humidity_2m, precipitation, wind_speed_10m

実装モジュール

- `src/data/weather_client.py` — API クライアント
- `tests/data/test_weather_client.py` — テスト（モックベース）

実装内容

- `WeatherClient` Protocol を定義
- `OpenMeteoClient` クラスで実装
- リトライ（最大3回）・タイムアウト（30秒）を組み込み
- レスポンスを `pandas.DataFrame` に変換

実行コマンド

```
uv run pytest tests/data/test_weather_client.py -v
```

次のステップ

- 電力需要データの取得・整形
- 気象データと電力データの結合
- 欠損値・外れ値の確認

データ変換・品質管理の実装

状態: ✅ Done

概要

- 欠損値補完（線形補間）
- 外れ値検出（IQR 法）
- 時系列整合性チェック
- 学習用データセット構築

実装モジュール

- `src/data/preprocessing.py` — 変換・品質管理
- `tests/data/test_preprocessing.py` — テスト

実装内容

- `fill_missing_values()` — 線形補間で欠損値を補完
- `detect_outlier_iqr()` — IQR 法で外れ値を検出
- `check_temporal_consistency()` — 時系列の整合性チェック
- `build_dataset()` — 気象データと電力データを結合し学習用データセットを構築

実行コマンド

```
uv run pytest tests/data/test_preprocessing.py -v
```

次のステップ

- 特徴量エンジニアリング（時間特徴・ラグ・ローリング統計）

特徴量エンジニアリングの実装

状態: ✅ Done

概要

- 時間特徴（hour, day_of_week, month, is_weekend）
- ラグ特徴（1h, 24h, 168h）
- ローリング統計（mean, std, 24h窓）

実装モジュール

- `src/features/feature_engineering.py` — 特徴量エンジニアリング
- `tests/features/test_feature_engineering.py` — テスト

実装内容

- `add_time_features()` — 時間特徴を追加
- `add_lag_features()` — ラグ特徴を追加
- `add_rolling_features()` — ローリング統計を追加
- `build_features()` — 全特徴量を一括追加

実行コマンド

```
uv run pytest tests/features/test_feature_engineering.py -v
```

次のステップ

- LightGBM モデルの学習・評価

LightGBM モデルの学習・評価

状態: ✅ Done

概要

- LightGBM 回帰モデルの学習
- 時系列 train/test 分割（80/20）
- MAE / RMSE / MAPE による評価
- 特徴量重要度の確認

実装モジュール

- `src/models/lgbm_model.py` — モデル学習・評価
- `tests/models/test_lgbm_model.py` — テスト

実装内容

- `train_test_split()` — 時系列を保持した分割
- `train()` — LightGBM 回帰モデルの学習
- `evaluate()` — MAE / RMSE / MAPE を計算
- `feature_importance()` — 特徴量重要度を DataFrame で取得

実行コマンド

```
uv run pytest tests/models/test_lgbm_model.py -v
```

次のステップ

- Optuna によるハイパーパラメータ最適化
- 時系列クロスバリデーション（TimeSeriesSplit）
- 学習曲線の可視化

Optuna ハイパーパラメータ最適化

状態: ✅ Done

概要

- Optuna TPESampler による自動探索
- 時系列クロスバリデーション（TimeSeriesSplit）
- 最適パラメータの保存（JSON）
- 学習曲線・パラメータ重要度の可視化

実装モジュール

- `src/models/optuna_tune.py` — ハイパーパラメータ最適化
- `tests/models/test_optuna_tune.py` — テスト

実装内容

- `objective()` — Optuna 目的関数（TimeSeriesSplit CV）
- `tune()` — 最適パラメータ探索を実行
- `save_best_params()` — 最適パラメータを JSON 保存
- `plot_optimization_history()` — 学習曲線を可視化
- `plot_param_importances()` — パラメータ重要度を可視化

実行コマンド

```
uv run pytest tests/models/test_optuna_tune.py -v
```

次のステップ

- 最適パラメータを lgbm_params.yaml に反映
- 最終モデルの再学習・保存
- 推論 API の実装

推論 API の実装

状態: ✅ Done

概要

- FastAPI による REST API
- 最適パラメータ済みモデルの推論
- ヘルスチェック・バージョン情報エンドポイント

実装モジュール

- `src/serving/app.py` — FastAPI アプリ
- `tests/serving/test_app.py` — テスト

実装内容

- `GET /health` — ヘルスチェック
- `GET /version` — モデルバージョン情報
- `POST /predict` — 単一予測
- `POST /predict/batch` — バッチ予測

実行コマンド

```
uv run pytest tests/serving/test_app.py -v
uv run uvicorn src.serving.app:app --reload --port 8000
```

次のステップ

- Docker コンテナ化
- 本番デプロイ（Cloud Run / Fly.io）

Docker コンテナ化

状態: ✅ Done

概要

- マルチステージビルド
- uv による高速依存関係インストール
- 非 root ユーザー実行
- ヘルスチェック組み込み

実装ファイル

- `Dockerfile`
- `.dockerfile.dockerignore`

実行コマンド

```
docker build -t power-demand-forecast:latest .
docker run -p 8000:8000 -v ./models:/app/models:ro -v ./data:/app/data:ro power-demand-forecast:latest
docker image ls power-demand-forecast
```

次のステップ

- GitHub Actions CI/CD
- 本番デプロイ（Cloud Run / Fly.io）

GitHub Actions CI/CD

状態: ✅ Done

概要

- lint / type-check / test / security-audit の 4 ジョブ
- uv キャッシュによる高速化
- pip-audit による脆弱性スキャン

実装ファイル

- `.github/workflows/ci.yaml`

次のステップ

- 本番デプロイ（Cloud Run / Fly.io）
- Docker イメージの自動ビルド・push

本番デプロイ（Cloud Run）

状態: ⏸ Pending

前提条件: GCP プロジェクト設定

実装状況サマリー

モジュール	ステータス	テスト
データ基盤	✅ Done	—
Open-Meteo 気象データ API クライアント	✅ Done	✅
気象データ変換モジュール	✅ Done	✅
電力需要データ読み込み	✅ Done	✅
統合データセット構築パイプライン	✅ Done	✅
欠損値補完	✅ Done	✅
外れ値検出（IQR 法）	✅ Done	✅
時系列整合性チェック	✅ Done	✅
特徴量エンジニアリング	✅ Done	✅
LightGBM モデル学習・評価	✅ Done	✅
Optuna ハイパーパラメータ最適化	✅ Done	✅
推論 API	✅ Done	✅
Docker コンテナ化	✅ Done	—
GitHub Actions CI/CD	✅ Done	—
本番デプロイ（Cloud Run）	⏸ Pending	—

開発ツール

コードの品質管理

Linter: ruff v0.3.0 **フォーマッタ:** ruff v0.3.0 (Black互換モード)

型チェック: mypy v1.8.0

pre-commit フック: trailing-whitespace, end-of-file-fixer, check-yaml, check-toml, check-ast, check-merge-conflict, check-large-files (1MB), debug-statements, mixed-line-ending, ruff, ruff-format, mypy

依存関係管理

- pyproject.toml で依存関係を宣言
- uv.lock でロックファイルを管理

開発コマンド

```
uv sync --dev                    # 開発環境のセットアップ
uv run ruff check .              # Lint
uv run ruff format --check .     # フォーマットチェック
uv run mypy src/                 # 型チェック
uv run pytest                    # テスト
uv run pre-commit run --all-files # 全フック実行
```

開発ツール

ツール	バージョン	用途
ruff	0.3.0	Lint + フォーマット
mypy	1.8.0	型チェック
pre-commit	3.6.0	Gitフック管理
pytest	8.0.0	テスト
pytest-cov	4.1.0	カバレッジ測定

CI/CD

ジョブ	ツール	実行内容
Lint	ruff	check + format --check
Type Check	mypy	型チェック
Test	pytest	ユニットテスト + カバレッジ
Security	pip-audit	脆弱性スキャン

実装原則

- **Protocol ベースの抽象化** — 継承ではなく構造的サブタイピング
- **ファイル単位の分離** — 1 ファイル = 1 クラス
- **TDD** — テストを先に書く
- **型ヒント必須** — mypy strict モード
- **ruff 設定** — E, W, F, I, N, UP, B, SIM, RUF ルール

ディレクトリ構成

```
src/
tests/
pyproject.toml
uv.lock
.pre-commit-config.yaml
```

コマンド

```
uv run pytest -v -s -x --cov=src --cov-report=term-missing
uv run ruff check --fix .
uv run ruff format .
uv run mypy src/
uv run pre-commit install
uv run pre-commit run --all-files
```

データ基盤

データ取得基盤の実装

状態: ✅ Done

概要

- Open-Meteo 気象データ API クライアント実装
- 気象データ変換モジュール実装（DataFrame 形式）
- 電力需要データ CSV 読み込み実装
- 統合データセット構築パイプライン実装

データ取得の仕様

状態: ✅ Done

概要

- Open-Meteo 気象データ API の仕様調査
- 対象地域・変数・期間の定義

データ変換・品質管理の仕様

状態: ✅ Done

概要

- 欠損値補完（線形補間）
- 外れ値検出（IQR 法）
- 時系列整合性チェック
- 学習用データセット構築

実装モジュール

- `src/data/preprocessing.py` — 変換・品質管理
- `tests/data/test_preprocessing.py` — テスト

実装内容

- `fill_missing_values()` — 線形補間で欠損値を補完
- `detect_outlier_iqr()` — IQR 法で外れ値を検出
- `check_temporal_consistency()` — 時系列の整合性チェック
- `build_dataset()` — 気象データと電力データを結合し学習用データセットを構築

実行コマンド

```
uv run pytest tests/data/test_preprocessing.py -v
```

次のステップ

- 特徴量エンジニアリング（時間特徴・ラグ・ローリング統計）

特徴量エンジニアリングの実装

状態: ✅ Done

概要

- 時間特徴（hour, day_of_week, month, is_weekend）
- ラグ特徴（1h, 24h, 168h）
- ローリング統計（mean, std, 24h窓）

実装モジュール

- `src/features/feature_engineering.py` — 特徴量エンジニアリング
- `tests/features/test_feature_engineering.py` — テスト

実装内容

- `add_time_features()` — 時間特徴を追加
- `add_lag_features()` — ラグ特徴を追加
- `add_rolling_features()` — ローリング統計を追加
- `build_features()` — 全特徴量を一括追加

実行コマンド

```
uv run pytest tests/features/test_feature_engineering.py -v
```

次のステップ

- LightGBM モデルの学習・評価

LightGBM モデルの学習・評価

状態: ✅ Done

概要

- LightGBM 回帰モデルの学習
- 時系列 train/test 分割（80/20）
- MAE / RMSE / MAPE による評価
- 特徴量重要度の確認

実装モジュール

- `src/models/lgbm_model.py` — モデル学習・評価
- `tests/models/test_lgbm_model.py` — テスト

実装内容

- `train_test_split()` — 時系列を保持した分割
- `train()` — LightGBM 回帰モデルの学習
- `evaluate()` — MAE / RMSE / MAPE を計算
- `feature_importance()` — 特徴量重要度を DataFrame で取得

実行コマンド

```
uv run pytest tests/models/test_lgbm_model.py -v
```

次のステップ

- Optuna によるハイパーパラメータ最適化
- 時系列クロスバリデーション（TimeSeriesSplit）
- 学習曲線の可視化

Optuna ハイパーパラメータ最適化

状態: ✅ Done

概要

- Optuna TPESampler による自動探索
- 時系列クロスバリデーション（TimeSeriesSplit）
- 最適パラメータの保存（JSON）
- 学習曲線・パラメータ重要度の可視化

実装モジュール

- `src/models/optuna_tune.py` — ハイパーパラメータ最適化
- `tests/models/test_optuna_tune.py` — テスト

実装内容

- `objective()` — Optuna 目的関数（TimeSeriesSplit CV）
- `tune()` — 最適パラメータ探索を実行
- `save_best_params()` — 最適パラメータを JSON 保存
- `plot_optimization_history()` — 学習曲線を可視化
- `plot_param_importances()` — パラメータ重要度を可視化

実行コマンド

```
uv run pytest tests/models/test_optuna_tune.py -v
```

次のステップ

- 最適パラメータを lgbm_params.yaml に反映
- 最終モデルの再学習・保存
- 推論 API の実装

推論 API の実装

状態: ✅ Done

概要

- FastAPI による REST API
- 最適パラメータ済みモデルの推論
- ヘルスチェック・バージョン情報エンドポイント

実装モジュール

- `src/serving/app.py` — FastAPI アプリ
- `tests/serving/test_app.py` — テスト

実装内容

- `GET /health` — ヘルスチェック
- `GET /version` — モデルバージョン情報
- `POST /predict` — 単一予測
- `POST /predict/batch` — バッチ予測

実行コマンド

```
uv run pytest tests/serving/test_app.py -v
uv run uvicorn src.serving.app:app --reload --port 8000
```

次のステップ

- Docker コンテナ化
- 本番デプロイ（Cloud Run / Fly.io）

Docker コンテナ化

状態: ✅ Done

概要

- マルチステージビルド
- uv による高速依存関係インストール
- 非 root ユーザー実行
- ヘルスチェック組み込み

実装ファイル

- `Dockerfile`
- `.dockerfile.dockerignore`

実行コマンド

```
docker build -t power-demand-forecast:latest .
docker run -p 8000:8000 -v ./models:/app/models:ro -v ./data:/app/data:ro power-demand-forecast:latest
docker image ls power-demand-forecast
```

次のステップ

- GitHub Actions CI/CD
- 本番デプロイ（Cloud Run / Fly.io）

GitHub Actions CI/CD

状態: ✅ Done

概要

- lint / type-check / test / security-audit の 4 ジョブ
- uv キャッシュによる高速化
- pip-audit による脆弱性スキャン

実装ファイル

- `.github/workflows/ci.yaml`

次のステップ

- 本番デプロイ（Cloud Run / Fly.io）
- Docker イメージの自動ビルド・push

本番デプロイ（Cloud Run）

状態: ⏸ Pending

前提条件: GCP プロジェクト設定

実装状況サマリー

モジュール	ステータス	テスト
データ基盤	✅ Done	—
Open-Meteo 気象データ API クライアント	✅ Done	✅
気象データ変換モジュール	✅ Done	✅
電力需要データ読み込み	✅ Done	✅
統合データセット構築パイプライン	✅ Done	✅
欠損値補完	✅ Done	✅
外れ値検出（IQR 法）	✅ Done	✅
時系列整合性チェック	✅ Done	✅
特徴量エンジニアリング	✅ Done	✅
LightGBM モデル学習・評価	✅ Done	✅
Optuna ハイパーパラメータ最適化	✅ Done	✅
推論 API	✅ Done	✅
Docker コンテナ化	✅ Done	—
GitHub Actions CI/CD	✅ Done	—
本番デプロイ（Cloud Run）	⏸ Pending	—